

Exame de Paradigmas de Programação — Época de Recurso / Modelo 1

|||

|---|---|

Instituição	P.PORTO — Escola Superior de Tecnologia e Gestão
Tipo de Prova	Exame Escrito — Época de Recurso / Modelo 1
Curso	Licenciatura em Engenharia Informática / Licenciatura em Segurança Informática em Redes de Computadores
Unidade Curricular	Paradigmas de Programação
Ano Letivo	2025/2026
Duração	2 horas

Observações

- Não é permitida a consulta.
- Não são permitidas questões relativas à Parte 2. Sempre que considerarem necessário, os alunos devem assumir os pressupostos que entenderem adequados, indicando-os explicitamente na resolução.

Parte 1

Pergunta 1

1,5 val

Explique detalhadamente a diferença entre Sobrecarga (*Overloading*) e Sobreposição (*Overriding*) de métodos em Java. Em que situações ocorre cada um e qual a relação destes conceitos com o Polimorfismo (estático e dinâmico)? Justifique a sua resposta e ilustre com um exemplo prático de cada mecanismo.

Pergunta 2

1,5 val

Descreva o mecanismo de Tratamento de Exceções em Java, distinguindo claramente entre *Checked Exceptions* e *Unchecked Exceptions* (como `RuntimeException`). Quando é que um programador deve optar por criar e lançar cada um destes tipos de exceção? Fundamente a sua explicação com exemplos concretos aplicados a cenários reais.

Pergunta 3

1,5 val

No paradigma Orientado a Objetos, a reutilização de código pode ser alcançada através da Herança ou da Composição. Explique ambos os conceitos e discuta o princípio de que se deve "favorecer a composição em detrimento da herança". Apresente cenários e um exemplo prático onde a composição é uma escolha de design mais adequada do que a herança.

Pergunta 4

1,5 val

Explique os conceitos de *Shallow Copy* (cópia superficial) e *Deep Copy* (cópia profunda) no contexto da instanciação e clonagem de objetos em Java. Discuta os potenciais problemas que podem surgir ao utilizar *Shallow Copy* em objetos que contêm referências para outros objetos mutáveis. Forneça um exemplo prático que demonstre a implementação correta de uma *Deep Copy*.

Parte 2

1. Considere as seguintes interfaces `Container` e `PerishableContainer`. A interface `Container` define o contrato para um contentor genérico utilizado nas caixas de suprimentos (AidBoxes). A interface `PerishableContainer` especializa `Container` e acrescenta operações específicas para contentores de alimentos perecíveis que requerem controlo de temperatura e de validade.

```
public interface Container {  
    String getCode();  
    double getCapacity();  
    ItemType getType();  
}
```

```
public interface PerishableContainer extends Container {  
    LocalDate getExpirationDate();  
    double getMinimumTemperature();  
    boolean isExpired(LocalDate currentDate);  
    boolean equals(Object obj);  
}
```

Pergunta 1a

3 val

Considere a interface `PerishableContainer` que representa um contentor de itens perecíveis. Implemente a interface numa classe denominada `PerishableContainerImpl`. O contentor deve possuir um **estado** (`ACTIVE`, `INACTIVE`) e deve ser inicializado como `ACTIVE` por defeito. Para a implementação do método `isExpired`, este deve retornar `true` se a `currentDate` fornecida por argumento for estritamente posterior à data de validade (`ExpirationDate`). Para a implementação do método `equals`, considere que **duas instâncias** de `PerishableContainer` são iguais se possuírem o mesmo código (devolvido através do método `getCode()`).

Pergunta 1b

2 val

Desenvolva o código necessário para testar a classe implementada (por exemplo, no contexto de um método `main`). Apresente um exemplo de teste que valide a instanciação, o funcionamento do método `isExpired` e a comparação de igualdade através do método `equals`.

2. Considere a existência de uma classe `CollectionManagerImpl` que implemente a interface `CollectionManager`.

```
public interface CollectionManager {  
    SupplyCollection[] createCollections(IInstitution inst);  
}
```

Pergunta 2a

4 val

Na classe `CollectionManagerImpl`, implemente os seguintes métodos auxiliares:

```
double calculateTotalLoad(AidBox aidbox, ItemType type);
```

- Este método deve calcular e devolver o somatório do valor da última medição (acedido através de `getLastMeasurement().getValue()`) de todos os contentores presentes na `AidBox` cujo tipo corresponda ao `ItemType` fornecido. Note que apenas deve contabilizar a carga dos contentores do tipo especificado.

```
boolean addAidBoxToCollection(SupplyCollection collection, AidBox aidbox, Vehicle vehicle);
```

- O método deve devolver `true` caso a `AidBox` seja adicionada com sucesso à recolha (`SupplyCollection`).
- Para adicionar a `AidBox`, o método deve primeiro calcular a carga que será adicionada (utilizando o método `calculateTotalLoad` para o tipo de item do veículo).
- De seguida, deve validar se esta carga, somada à carga já existente na recolha (obtida através de `collection.getCurrentLoad()`), **não excede** a capacidade máxima do veículo (`vehicle.getMaxCapacity()`).
- Se a validação for bem-sucedida, a `AidBox` deve ser adicionada à recolha utilizando o método `void addAidBox(AidBox aidBox)` da classe `SupplyCollection`.
- Caso a invocação do método `addAidBox` origine uma `SupplyCollectionException`, o método `addAidBoxToCollection` deve retornar `false`.

Pergunta 2b

5 val

Na classe `CollectionManagerImpl`, implemente o método `createCollections`, gerando as recolhas necessárias com base nas regras estabelecidas.

Regras a considerar:

- Para cada veículo devolvido pelo método `getVehicles()` da interface `IInstitution`, deve ser criada uma nova recolha (`SupplyCollection`). Assuma a existência do construtor `SupplyCollectionImpl(Vehicle v)`. Assuma que só existe um veículo para cada tipo.
- As `AidBoxes` existentes para recolha são as devolvidas pelo método `getAidBoxes()` da interface `IInstitution`.
- Deve iterar sobre todas as `AidBoxes` e tentar adicioná-las à `SupplyCollection` correspondente ao seu veículo (veículo com o mesmo `ItemType` dos contentores a recolher).
- Deve utilizar os métodos desenvolvidos na alínea anterior. Se não os implementou anteriormente, assuma que os métodos já existem.
- O array devolvido pelo método `createCollections` deve ser perfeitamente dimensionado, contendo apenas as recolhas que possuam **pelo menos uma** `AidBox` (ou seja, sem posições nulas no array ou recolhas sem caixas).

Excertos de Código Fornecidos

Para além dos excertos de código fornecidos, considere o seguinte resumo dos métodos para a resolução do exercício:

```
public interface IInstitution {
    Vehicle[] getVehicles();
    AidBox[] getAidBoxes();
}

public interface AidBox {
    Container[] getContainers();
}

public interface Container {
    ItemType getType();
    double getCapacity();
    Measurement getLastMeasurement();
}

public interface Measurement {
    double getValue();
}

public interface Vehicle {
    String getCode();
    ItemType getSupplyType();
    double getMaxCapacity();
}

public interface SupplyCollection {
    Vehicle getVehicle();
    void addAidBox(AidBox aidBox) throws SupplyCollectionException;
    double getCurrentLoad();
    AidBox[] getAidBoxes();
}
```